The bench harness

Three gates, each roughly ten times the cost of the one before it. Run them in order; a cheap failure stops an expensive one.

gate	cost	needs	what it answers
`tools/release_sweep.py --offline`	~1 min	nothing	can this code possibly work
`tools/bench_smoke.py`	12.8 min	both instruments	is the bench and the harness sound
`tools/soak.py --suites formats,plan`	~3.0 h a lap	both instruments	does it hold up over hours

Measured, not estimated: **6.5 s per cell**. A lap of 39 waveforms at 43 rates each is 1677 cells, so **about 3.0 hours**; all 41 waveforms is 3.2 h. An offline lap of the same cells is **about 4 seconds** at one capture offset per cell and **19 seconds** at eight, measured on 12 workers.

Eight hours is therefore between two and three laps. That is the floor, not a target: the point of the soak is a failure rate per point, and two laps can only ever say "twice" or "once" or "never".

The seeded sweep

`tools/soakplan.py` decides what one iteration tests. Everything comes from the iteration number, so **iteration 129 is reachable without running 128 first**.

Per iteration, four things vary and nothing else does:

- the **order** waveforms load in, so a state leak out of one stops looking like a defect in whichever waveform always follows it
- the **non-standard rate** drawn in each gap between standard rates
- the **wait** before each capture, per cell
- the **vertical placement** of each cell: a target logic swing drawn uniformly from **0.45 V to 8 V**, and a DC offset drawn from what is left inside the rails

The vertical draw, and the one region it will not use

The swing is asked for **in volts**, not as a scale factor. Scaling each vector's own rendered span bottoms out at 1.65 V p-p on the 3.3 V vectors, which left the bottom of the claimed range untested by construction; asking for a span in volts covers 0.45 V to 8 V on every vector that can reach it, and reports the achieved figure per row so a generator ceiling shows up as a ceiling rather than as coverage. A 3.3 V-span vector needs 24.2 Vpp for an 8 V swing and the SDG stops at 20, so its achieved span caps at 5.28 V — stated, not silently absorbed.

The offset draw skips the window that would put a single-supply band across ground, and that exclusion is not cosmetic. `sig_levels` decides idle polarity from the levels when no run in the window reaches ten bit times: `lo < -flatfloor and hi > flatfloor` means RS-232, which marks at its *negative* level. So a 0…6.6 V logic waveform shifted until its low is below −1 V and its high above +1 V is, to the app, correctly read as an inverted line — right rate, right format, self-consistent bytes matching nothing, one frame short. Unconstrained, **17.3 % of a lap's cells land in that window and they account for 86.8 % of all offline failures** — enough to put the true rate at ~0.23 % behind a measured 1.79 %, and enough to manufacture a "failures rise with swing" gradient, since a large span is what lets a shifted band clear 1 V on both sides.

Only the straddling window is cut, and this matters: raising the lower bound instead also removes the legitimate wholly-negative region, ~57.5 % of the interval on some vectors, where an offset of –6.5 V decodes while –3.6 V does not. `soakplan.py --selftest` asserts the span range, that the draw reaches both ends of it, that nothing clips, and that nothing straddles ground. `FLATFLOOR` is **read out of `tsp/serial_core.tsp`** rather than copied, so the constraint cannot drift away from the rule it exists to respect.

Fixed: the 22 standard baud rates in `[300, 250000]`, tested on every waveform every iteration. `250000` is `sdec.maxbaud`; below 300 costs 8 s of capture for a rate nothing here speaks.

Frame mode only. Above 165563 Bd `sdec.fs_for_burst` returns nil, so the streaming paths cannot record 172800 and up at all, and only frame reaches 250000.

Why the wait is scaled, not constant

Uniform over **10 byte-times** — which is 100 bit-times — so one continuous draw randomises the byte a capture lands on *and* the bit *and* the phase within the bit, because it is quantised to none of them. Scaled by baud it costs 0.4 min across a whole sweep where a flat 1 s costs 14.7.

It also lands where it is needed. Below about 20 kBd, socket jitter is smaller than a byte time and cannot move the byte offset at all; above it, jitter alone already scatters the phase by tens of bytes.

Why a uniform draw is not the only rule

`sdec.pick_fs` snaps **up** through a coarse ladder, so below 125 kBd samples-per-bit is a sawtooth whose minimum in each step sits at `baud = floor(fs/8)` — 312, 625, 1250 … 80000, 125000. Those are where the decoder is closest to being confidently wrong, and a uniform draw lands on one with probability about zero. A gap containing an edge therefore yields that edge one time in three. Above 125 kBd fs is pinned at 1 MSa/s and samples-per-bit falls smoothly to 4.00, so there is no cliff and uniform is right.

The smoke gate

Four waveforms, then the button matrix.

```
standard rates   v77  the fox, 8N1, 133 B repeating
                 r06  random,  7E1, 256 B non-repeating
drawn rates      v78  the fox, 7E1
                 r00  random,  8N1
```

Paired crosswise on purpose. Each rate family faces both formats and both content classes. Pairing by family instead — fox on the standard ladder, random on the drawn rates — would leave the standard rates never seeing a non-repeating payload, and that is where head alignment and the parity vote get tested.

86 cells in 9.5 min, then 45 presses in 1.9 min. `--plan-spec vector:std|nonstd|all` is what makes the quarter-coverage expressible; a cell's wait is still keyed on its index in the **full** rate list, so a cell reached this way is the same cell the soak runs.

Then two stages of about a minute each. **`levels`** plays one waveform at 5 V, 3.3 V, 1.6 V, 1.0 V, 0.5 V, 0.33 V and 0.25 V of logic swing and checks the `LOGIC` cell, that `THRESH` landed within a quarter of the swing of mid-swing, and the bytes — the published range is 0.33 V to 8 V, and the plan stage drives its own *drawn* amplitude rather than the claimed floor, so nothing else in the gate tests it. **`rates`** runs eight wrong-lock cases from `bench_break`, which is the only stage where the lock CONTRADICTS the wire. `rates` goes last because it stubs `sdec.ua_badfrac` on the instrument, so a teardown that failed cannot quietly change another stage's verdict.

`--no-panel`, `--no-levels` and `--no-rates` each skip a stage, and each writes a **zero** into the receipt for what it skipped, so a partial run cannot later be read as a full one.

The receipt

A pass writes `out/smoke-receipt.json` holding a hash of `tsp/` and `tools/`. `tools/hooks/pre-push` recomputes that hash and refuses a push when either tree has moved, so "I ran the smoke test" becomes a claim about the exact code being pushed rather than about the past. The hash folds in the working tree, not just the index, so an unstaged edit invalidates it too.

`SMOKE_OVERRIDE="reason" git push` proceeds and prints the reason. Docs-only pushes need no receipt.

Changing your mind mid-run

`--hours` and `--laps` are both decided before the first lap, so "make that four laps, not three" would otherwise cost a restart — and a restart throws away the laps already banked, which is the opposite of what one-more-lap means. Two files in the record directory are read before each lap:

```
echo 4 > <record dir>/LAPS      # run exactly this many laps, then stop
touch  <record dir>/STOP        # stop cleanly after the lap in flight
```

Neither interrupts a lap. **A lap is the unit of evidence** — a truncated one is not tallied, so cutting one wastes every minute already spent on it. For the same reason the wall deadline from `--hours` only gates *starting* a lap: one that begins inside the budget always runs to completion.

The release gate, stage by stage

`tools/release_sweep.py` runs every check in one go and writes an auditable record to `out/release/<timestamp>/`: a `REPORT.md`, a `summary.json`, every stage's full output, every front-panel screenshot, and the exact `.tspa` and `MANUAL.pdf` the results describe with their SHA-256s. It exits non-zero if any gate fails. `--offline` skips every stage needing an instrument and runs in seconds; `--skip name,name` drops named stages, and a skipped gate is reported as *not passed* rather than passed.

The full sweep needs a freshly power-cycled DMM6500. `sdec.start()` builds the display objects and the app refuses a second build in one power cycle, so the sweep checks that up front rather than failing forty minutes in. It also needs the SDG2122X loaded with the stimulus waveforms (`bench_matrix.py --upload`), and only one client may hold the DMM's control socket at a time.

Stage	Checks
`lint` `parse`	Lua 5.0.2 incompatibilities; `luac -p` on every module
`vecrefs`	every waveform id named in `tools/` is in `MAP` or declared `RETIRED` — deleting from `MAP` alone leaves references that fail where they are used, not where they are declared
`soakrand`	`mt19937.py` and `mt19937.lua` produce one sequence, checked against CPython's `random`, plus the 5.0.2 scan deciding whether the module can load on the instrument

Stage	Checks
`unit`	1 194 offline tests — decoder, UI, state machine, file paths, every visible ASCII glyph
`unit-cancel`	one-press recordings, both window sizes, and the flow-control loop with no interaction
`unit-frontrig`	TRIGGER-key and rear-BNC arming *through* `acquire()` — the routing that can drop to free-run while the status row still claims the key is the source
`unit-usblog`	USB log name allocation and the persistent index: exhaustion must refuse rather than loop and hang the panel
`unit-stream`	the streaming arm — both 4915 defences, one press per slice rather than per window
`unit-patterns`	byte-exactness on the hard payloads: edge-density extremes, walking bits and known random data, at the sample rates the panel actually picks
`unit-forcerate`	a forced rate the wire does not carry must **refuse** and name the rate it does carry, with both answers drivable unattended
`unit-judgev`	the harness's three verdicts — too short to read is **inconclusive**, not silently wrong; an alignment survives one bad byte; a loud vector may miss a bounded few. Each with the wrong capture that must still fail
`unit-ratefit`	the #46 rate-misfit window, pinned per cell and per capture phase against a recorded table — every cell that reproduces must reproduce at every phase, and every cell that does not must be silent at all of them
`unit-plansweep`	the offline twin against the soak's own drawn plan, ratcheted on eleven counters at the configuration each baseline was measured at — iteration, offset count **and skipped set**, because all three change the draw
`unit-soaklog`	the soak's own lap log replayed through the completeness tests, so a lap that recorded nothing cannot read as a lap that found nothing
`stress`	hostile signals: never silently **wrong**, never **raises**
`unit-analog`	the bench cases at the app's own sample rates, swept over sampling phase, jitter, noise and where the window opens
`unit-phasesweep`	every vector × capture start × phase/jitter/noise, sharded across the cores — no raise, no result without a format, no wrong byte among trustworthy calls
`unit-seam`	a capture whose arb loop seam lands late must still be judged, and the narrower trim still required
`unit-sdgguard`	every route by which an out-of-spec waveform could reach the generator refuses it
`unit-loremgate`	the harness's own long-payload verdict: every clean run validated, flag count bounded
`tolerance`	recomputes the envelope table printed in the manual
`package archive`	rebuilds the `.tspa`, then builds both screens *from the archive* against a mock front end
`manual`	rebuilds every shipped PDF: `README.pdf`, `MANUAL.pdf`, `REFERENCE.pdf`, `BENCH.pdf`
`hw-matrix`	through the app's own Capture button: six frame formats, the standard rate ladder, a 1 kB non-repeating payload, **seven logic swings from 5 V down to 0.25 V**, twelve DC offsets

Stage	Checks
`hw-payloads`	fourteen payloads covering every byte value 0–255, two of them also at 115 200 and 250 000
`hw-odd-rates`	nineteen **non-standard** baud rates — 900, 1500, 3600, 8123, 29127, 104857 …
`hw-panel`	every button in every state, including a one-press recording. Six checks per press: the handler does not raise, logs no instrument event, returns inside its latency budget, reports what it did, changed the state it was supposed to, and **the panel actually shows it** — grabbed before and after each press and differenced by region
`soakrand-dmm`	the third leg: the instrument's **own** Lua 5.0.2 must produce the same words, floats, rejected draws and permutation
`hw-plan`	the seeded sweep on the bench: every waveform across all 43 rates in a seeded order, with a seeded wait, amplitude and DC offset drawn per cell. `--plan-vectors` cuts it to a subset for a lap that must finish in minutes. The vertical draw asks for a **target swing in volts, 0.45 V to 8 V**, and places the offset wherever keeps both levels inside ±9.5 V *and* does not put a single-supply band across ground. Every row records the swing it drove and the `S/N` the app reported
`hw-break`	degenerate signals and contradictory settings — no signal, DC only, all-`0x00` / `0xFF` / `0x55`, a break, 60 mV of swing, 19 Vpp, rates past the ceiling, six wrong forced settings. A refusal with a reason passes; confident garbage does not

This table must list every stage `release_sweep.py` defines, or the published inventory of what a release passed is incomplete. The check:

```python
import re
code = set(re.findall(r"Stage\(\s*'([^']+)'", open('tools/release_sweep.py').read()))
doc  = {n for l in open('docs/BENCH.md') if l.startswith('| `')
          for n in re.findall(r'`([a-z0-9-]+)`', l.split('|')[1])}
assert not (code - doc), sorted(code - doc)
```

Reproducing a failure

A failing cell prints a `REPRO` line carrying everything needed:

```
REPRO iteration 1 vector v48a rate 2400 Bd (std) srate 24000 spb 10 wait 38.5438 ms
      measured-head - nf 2 fmt 8N1 want 8N1
```

Replay it offline, where there is no analogue path and no race:

```
python3 tools/soakplan.py --emit-lua --iteration 1 > /tmp/p.lua
lua tools/sweep_plan.lua --plan /tmp/p.lua --cell v48a:2400
```

Fails offline too → logic, deterministic, debuggable. **Passes offline** → the difference is the hardware: the DAC, the cable, the digitiser, or the capture phase that the seeded wait can only perturb.

What the seed does and does not reproduce

The seed reproduces the **schedule** — which rates, in which order, with what commanded wait. On hardware it cannot reproduce the realised capture **phase**, because that is a race the wait only disturbs. This is why a failure record carries the **measured** head alignment: that is what converts a hardware failure into an exact offline replay. Offline the phase is set explicitly, so it is exact by construction.

The offline twin

`tools/sweep_plan.lua` replays a plan cell for cell with no instrument: it reads the `.bin` the generator actually holds, converts codewords to volts at the amplitude the file was encoded for, resamples to the rate the app would pick, starts at the phase the seeded wait produces, and decodes.

Interpolation is **linear** between arb points, not a step: a DAC into a reconstruction filter presents slope, and the scope measured clean 0/3.26 V edges. `--hold` is the harsher zero-order-hold model, available as a robustness sweep.

Pass the bench lap's own `--skip-vectors` through, or the comparison is between two different experiments. `soakplan` applies the skip **before** the shuffle, so dropping two waveforms moves every remaining vector's index — and that index keys the amplitude, the offset *and* the wait for every cell. Every hardware lap skips `v95` and `v96`, so a twin lap emitted without them drove a different waveform in **all 1677 cells**: `v94` at 300 Bd was 20.000 Vpp at –8.220 V on the bench and 8.595 Vpp at +2.347 V offline. The emitted plan now carries its `skipped` set and `sweep_plan.lua` prints it, so a mismatch is visible in the log rather than only in the numbers.

```
python3 tools/plan_sweep.py --iteration 1 --skip-vectors v95,v96 --offsets 8
```

`badcells` is a union over capture phases and must never be set beside a bench lap's failing-cell count. With `--offsets 8` a cell is called bad if any *one* of eight phases failed; the bench takes one capture and asks once. Measured against soak lap 1: 65 cells fail here and passed on the bench, and not one of them fails at all eight phases — 41 fail at one of eight, 20 at two, none past five. Per *capture* this harness fails 1.09 % against the bench's 2.80 %, so it is not harsher; the union is. `badall` is the intersection — the cells no capture placement saves — and it is the cell figure the two harnesses can be compared on. Both of its two cells on that lap, `v94` at 3572 and at 124077 Bd, are bench failures too.

What it is not. It is not the app. `bench_matrix` presses the real Capture button and reads the real panel; this calls `sdec` directly, so it never exercises the two-pass probe, autolock, or anything the operator sees — and it never exercises waveform selection, which is why a selection fault shows up on hardware and passes here. In particular the bench digitises at `pick_fs(`the baud its own **probe** measured`)` and this digitises at `pick_fs(`the baud that was **commanded**`)`; on soak lap 1 those disagreed on 26 of 1385 cells, 1.9 %. It also judges more simply: the bytes must be a cyclic substring of the payload — save for the `loud` mismatch allowance below, which is `bench_uart`'s own number — where `bench_uart.judge_payload_v` additionally weighs flag budgets, coverage floors and head damage.

And it is laxer than the bench in three places, which is the asymmetry that is easy to miss because it runs the other way. A cell whose bytes are right and whose **reported baud** is outside `snaptol` fails on the bench and passes here, because `classify()` is only reached once the payload verdict is already BAD. The bench fails an inconclusive cell on an `exact` vector where this counts it as unjudgeable. And the bench checks the reported **format**, which this does not.

Its head bound is the app's own, not a constant. It trims `max(ua_edge_frames, ua_head_bad)` bytes and then tries a bounded run of further shifts, counting any shift it actually needed as `bleed` rather than absorbing it. A fixed trim cannot work: `ua_head_bad` reads 34 on `v77` at 19200 Bd, so a 12-byte constant failed byte-exact decodes at all 43 rates and manufactured 3377 failures across 20 laps.

What it is worth, measured over a whole run rather than sampled from one. A 172.3 min offline soak put 2818 plan runs of 1176 points through the shipping tree — 3 313 968 decodes, **321 a second** — against a hardware lap's 1683 cells in 2.9 h, which is 0.16 a second. That is **1989×**: those 2.9 offline hours are **238 days** of soak decodes, about 83 days an hour.

Take that rate from a completed run, not from the first few minutes. The same figure measured over the opening 11.7 min came out 548 a second — 71 % high — because the short suites cycle first and the seeded sweeps that dominate the wall clock had barely started.

And it is decode volume, not coverage. The paths listed above are covered 0× however long it runs. Lap 1 of a hardware soak put 15 distinct rate misreports on the board, of which 3 — `37422→38400`, `149527→153600`, `187477→192000` — were the `rstdC` shape at 1.024–1.027×, 20 % of the lap. Offline that route fired **0 times in 141 040 decodes**. Volume is not coverage, and that contrast is the whole argument for running both.

What a waveform is entitled to do

Two classes, in `soakplan.VECTOR_EXPECT`:

- **`exact`** — must decode byte-exact at every rate. Anything else is a defect.
- **`loud`** — may fail, and **must not be silently wrong**. Declining, or failing with flags raised, is a pass. Confident bytes that are not the payload fail for these too.

A `loud` vector may also miss a bounded number of bytes, and the bench says how many. In 8N1 a data sample that crosses into the neighbouring cell yields a wrong byte with framing intact and no parity to catch it, so nothing is flagged and a jitter vector cannot be held to zero — `bench_uart` measured `j20` losing 1–5 bytes in 256 at 172800–240959 Bd and allows `max(2, 3 %` of the judged body `)`. The offline twin allows the same — sized inside its shift loop, from the body it is actually comparing, so a shifted match cannot spend an allowance its longer unshifted body earned — and `tools/test_judge_v.py` *executes* the twin's `loud_budget` under `lua` and compares it with `bench_uart`'s expression at ten body sizes, so the copy cannot drift in its constants or in its arithmetic. Holding a `loud` vector to byte-exactness where the bench allows 3 % accounted for **11 of the 73 cells** the twin failed and the bench passed on soak lap 1 — seven `j20` cells the bench logged as "6 mismatched of 6 allowed" and the like, and four `v47`. Every byte the allowance forgives is counted as `loudmiss` and ratcheted, for the same reason `loudquiet` is: a tolerance nobody counts is one nobody can bound. An `exact` vector still gets zero.

A decline is a pass, and it is still counted. `sweep_plan.lua` tallies these as `loudquiet` — 17 a lap at one capture offset, 90 at eight — and `plan_sweep.py` ratchets that count at every measured configuration. Both halves matter: calling a correct refusal a failure is a harness inventing defects, measured at about 80 a lap when it was tried, while leaving it uncounted means a regression that suppressed *every* byte on *every* `loud` vector would move no counter at all and read as a completely unchanged run. The ratchet is deliberately one-directional: loud vectors starting to decode is not a defect, so only a rise is gated.

A RAISE IS NOT A DECLINE, and the licence to fail must not cover it. A decode that threw arrives with no result, which is indistinguishable from an honest refusal unless the raise flag itself is tested — so granting the `loud` pass without checking it leaves `raised` at zero while the decoder is throwing, and `raised` is the one counter that gates unconditionally at every capture placement.

`loud`: `v47` (spikes stacking to 9.3 V), `v48a` and `v48b` (drift), `j20` (20 % jitter), `v61`–`v63` (LIN carries framing violations no UART frame can contain).

`v48a` is the instructive one. Its 0.6 V drift is inside tolerance **at its native rate**; swept two decades away the drift-to-window ratio changes, the two logic levels stop being the two densest amplitudes, and the app reports `baseline unstable — only 8 % of samples sit at a logic level` and declines. Declining is the correct answer to an unreadable signal, so it counts as one.

Ambiguous framing, and why the oracle accepts two answers

`v90`, `v92` and `v94` are blocks of `00/FF/55/AA` and walking bits. In every one of those byte values **bit 7 carries exactly the parity a 7-bit reading would require**, so the wire is a valid 8N1 frame *and* a valid 7E1/7O1 frame, and nothing in the signal separates them. `v92`'s `exp_hex` is its `.txt` with bit 7 stripped — `80 → 00`, `FE → 7E`.

Which reading the app votes for shifts with the rate, so `v94` reads 8N1 at some rates and 7E1 at others, correctly both times. The judge therefore accepts **either** reading, and the format check stands down where there is more than one. Demanding either single answer fails a correct decode at every rate that chose the other.

The random numbers

`tools/mt19937.py` and `tools/mt19937.lua` must produce one sequence, because the hardware sweep is driven from Python and the offline suites draw in Lua. `tools/test_soakrand.py` holds three legs together:

```
CPython random  ->  mt19937.py  ->  host Lua  ->  the instrument's own Lua 5.0.2
```

CPython's `random` **is** this algorithm, so it is an independent implementation rather than a pasted table of numbers. The Lua side is arithmetic-only on doubles — 5.0.2 has no bitwise operators, no `%` and no `#`, and all three are parse errors there, so one occurrence stops the whole module loading whether the branch is taken or not. `mul32` splits its multiplier so no product exceeds 2^48; a direct `1812433253 * (2^32-1)` is 860× over 2^53 and would silently lose the low bits it carries forward.

Decisions are keyed — `{magic, iteration, purpose, index}` — not drawn from one running stream, so replaying a single cell does not depend on how many ran before it, including any that failed.

Instrument hazards

The generator's LAN service wedges, and there is no way to recover it remotely. No instrument on this bench has a smart plug, so every power cycle — a wedged SDG, the DMM's once-per-power-cycle UI build limit, a DMM control socket left held by a client that died — costs a human at the bench, and an unattended run that wedges is over until someone arrives. Two known triggers for the generator: a few consecutive large `WVDT` uploads, and sweeping SRATE across many rates on a very large stored waveform.

So a per-waveform health check asks both instruments whether they are still answering, using a Lua round trip rather than `*IDN?` — the SCPI parser can keep answering after the app is gone. On a silent instrument `bench_matrix` exits **3**, and `soak.py` ends the whole run on that rather than tallying laps against a dead bench. Exit 1 means "a cell failed" and a soak should carry on from it.

One client each. The DMM6500 accepts one controlling socket and the SDG one SCPI session; a second connection silently steals replies. `sdg_alive` must be passed an already-open handle or it reports a wedge that is not there.

Watching a long run

`soak.py` captures each lap's output rather than streaming it, so a 155-minute lap is otherwise opaque from outside. `bench_matrix --heartbeat FILE` appends a flushed, timestamped line as each waveform starts and finishes. A healthy run appends one every ~2.5 min:

```
2026-08-26T05:46:18 iter 1 DONE  32/41 v51 43 cells 246s 5.72s/cell 0 badcells 0 inconc
0 baud 0 events DMM=alive SDG=alive
```

Read the fields as:

field	meaning
`32/41` `v51`	waveform 32 of 41 this lap, and which one
`43 cells`	rate cells swept on this waveform
`0 badcells`	cells that did **not** decode as expected. Named to match the console's "not as expected"; double digits are normal for an impairment waveform, tagged `(loud, …)`, and for `v96`, which has an open issue
`0 inconc`	cells the judge declined — neither a pass nor a defect
`0 baud`	cells that misreported their rate
`0 events`	unexpected **instrument** events on this waveform. **This is the one that must be zero.** A `2205` or `2208` here means the app is filing file-system errors, which also pops them up on the panel
`DMM=` `SDG=`	whether each instrument still answered at the end of the waveform

`badcells` and `events` are different measurements and are easy to confuse: `events` counts the `*** unexpected event NNNN ***` lines, and those otherwise stay invisible until the lap ends, since the child's stdout is captured rather than streamed.

Do **not** use the child's CPU time as a liveness signal. The work is I/O bound at about 0.6 s of CPU an hour, and the child's pid changes at every lap boundary with its counter resetting to zero — so a monotonic check reports a stall at precisely the moment a lap completes. Judge liveness by the waveform number advancing and by the `DMM=`/`SDG=` footer: an `*IDN?` reply is not evidence, because the generator answers it with its waveform service wedged.

Every lap's per-cell output is written to `<record dir>/lap<n>-<verdict>.log` — including a lap the bench died under, tagged `STOPPED`. A lap that is not tallied is still examinable, which is what makes a later `grep` for an event code mean something.

Regrabbing the manual's figures

`tools/doc_shots.py` drives all nine panel screenshots in one pass and writes them into `docs/img`, which is version-controlled — so a bad grab replaces a shipped figure and the only way back is `git checkout`. Each shot therefore lands through a staging file renamed into place, and each is gated on three separate things:

check	asks	why it is not the others
`acted`	did the handler actually run	`ds_call` pcalls it and prints `ok=`. A refused press still leaves a screen to photograph.
`decoded`	did the capture produce bytes	only the frame shots consult a capture *shape*; a recording that got nothing has none.
`paged_ok`	is there really a page 2	`ui_page` is 0-based, so `page 0 of 1` passes any check that only reads the count.

A grab succeeding says the screen was read — not that anything was on it, and not that the press did anything. `--selftest` runs all three against nine recorded field sets with no instrument attached. `--only NAME[,NAME]` regrabs a subset, and `--retries` bounds the stochastic shapes: a mid-byte start is about one capture in eight, so the head shots retry.

A recording shot needs a locked rate, and clearing the lock is what makes the 32 kB shots come back empty. **Rewrite the captions from the new images rather than carrying them over** — a caption is a claim about a picture, and a stale one survives because nobody re-reads the figure. The trap when writing them: the paged figures are 32 kB recordings showing their retained **8 192-byte tail** (`sdec.ck_keep`), so the page counts are the tail's arithmetic, not the run's, and neither the byte count nor the page count can be read off the other.